

Implementacija i simulacija Istanbul Byzantine Fault Tolerance (IBFT) algoritma

Aleksandra Ilijoski E2 87/2025

I Uvod i motivacija

Istanbul Byzantine Fault Tolerance (IBFT) je algoritam konsenzusa koji je postao standard u *enterprise* blockchain rešenjima, od kojih je najpoznatiji **Quorum**.

Fokusiraćemo se na naučni rad objavljen 2020. godine koji je od ključnog značaja za ovu oblast. Naime, rane verzije IBFT-a (predložene kroz EIP-650) bile su zasnovane na neformalnim specifikacijama, što je dovodilo do ozbiljnih problema u praksi – mreža bi ponekad upadala u "mrtve petlje" (*deadlock*) ili bi dolazilo do narušavanja integriteta podataka. Rad koji analiziramo donosi prvu formalnu specifikaciju i dokazuje ispravnosti, rešavajući ove kritične mane i čineći IBFT stabilnim za finansijske sisteme gde greška nije opcija.

Zašto IBFT, a ne nešto drugo? U javnim mrežama poput Bitkoina, Proof of Work je neefikasan za zatvorene sisteme zbog velike potrošnje energije i sporosti. U privatnim ili konzorcijskim mrežama, gde su identiteti validatora poznati, BFT algoritmi su prirodno rešenje jer nude instantnu finalnost (nema "čekanja" na potvrde kao kod Bitkoina).

II Opis IBFT algoritma

IBFT vuče korene iz čuvenog PBFT rada (Castro-Liskov, 1999). On koristi trofazni proces glasanja: **Pre-prepare**, **Prepare** i **Commit**.

Pre nego što uđemo u detalje, definisaćemo ključne pojmove:

- **Validator:** Čvor koji učestvuje u procesu postizanja konsenzusa.
- **Proposer (Predlagač):** Validator izabran da predloži novi blok u određenoj rundi.
- **Round (Runda):** Pokušaj da se postigne dogovor unutar jedne sekvence. Ako predlagač ne uspe da prikupi dovoljno glasova, runda se povećava i bira se novi lider.
- **Sequence (Sekvenca):** Predstavlja visinu blockchain-a (redni broj bloka koji pokušavamo da dodamo).
- **Backlog:** Privremeno skladište za poruke koje su stigle "pre vremena" (npr. iz budućih rundi ili sekvenci).
- **Consensus proof (Dokaz konsenzusa):** Skup digitalnih potpisa koji dokazuju da je blok prošao kroz sve faze i da je validan.
- **Round state (Stanje runde):** Poruke konsenzusa specifične sekvence i runde, uključujući pre-prepare, prepare i commit poruku.

- **Snapshot (Snimak stanja):** Stanje glasanja validatora iz poslednje epohe (epoch).
- **Proposal (Predlog):** Predlog generisanja novog bloka koji prolazi kroz procesiranje konsenzusa.

Formalni zapis poruka

Sve poruke u protokolu su predstavljene kao torke (skupovi podataka). Postoje **četiri tipa poruka** koji su podeljeni u dve strukturne kategorije:

- **Uobičajene poruke:** $\langle tip, \lambda, r, value \rangle$ – gde je λ instanca (visina bloka), r je runda, a $value$ je predlog. Ovde spadaju PRE-PREPARE, PREPARE i COMMIT.
- **Poruka promene runde:** $\langle ROUND-CHANGE, \lambda, r, pr, pv \rangle$ – sadrži dodatne informacije o "pripremljenoj rundi" (pr) i "pripremljenoj vrednosti" (pv), što omogućava progres mreže čak i kada trenutni lider zataji.

Logika kvoruma (Matematika sistema)

Sistem je dizajniran da toleriše do F malicioznih (neispravnih) čvorova. Da bi to bilo moguće, ukupna mreža mora imati najmanje $N=3F+1$ validatora.

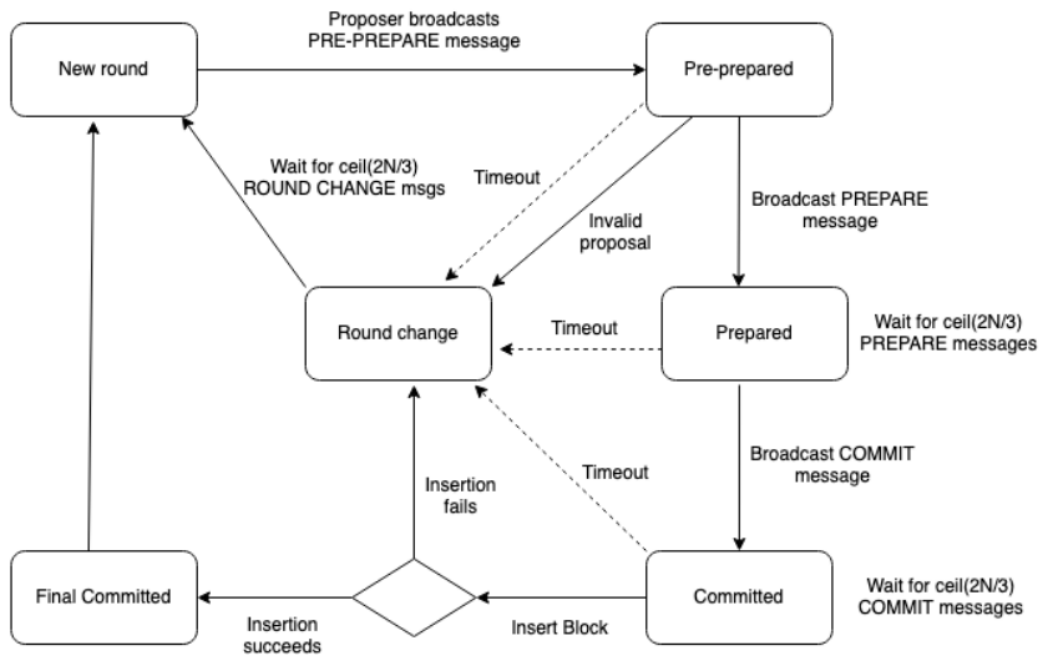
- **Zašto za COMMIT (odluku) treba $2F+1$ glasova?** Ovo osigurava **Safety** (Sigurnost). Matematički, ako imamo $3F+1$ čvorova i zahtevamo kvorum od $2F+1$, bilo koja dva takva kvoruma moraju da se preklapaju u bar $F+1$ čvorova. Pošto znamo da je maksimalan broj lažova F , bar jedan pošten čvor se nalazi u oba kvoruma. Taj čvor je "garant" koji nikada ne bi glasao za dve različite odluke, čime se sprečava razdvajanje mreže (*fork*).
- **Zašto za ROUND-CHANGE (promenu runde) treba samo $F+1$ glasova?** Ovo osigurava **Liveness** (da sistem ne stane). Ako bi lider bio maliciozan i prestao da šalje blokove, sistem mora da krene dalje. Pošto lažova ima najviše F , ako dobijemo $F+1$ zahteva za promenu lidera, to je siguran signal ("alarm") da je bar jedan pošten čvor primetio problem. To je dovoljno da i ostali pošteni čvorovi prihvate promenu.
- **Round-Robin selekcija:** IBFT koristi najjednostavniji sistem rotacije. Validatori su poređani u krug; ako lider runde 0 ne uspe, lider runde 1 je sledeći u nizu. Ovo sprečava bilo kog pojedinca da trajno blokira mrežu.

KAKO ALGORITAM RADI

U svojoj suštini, Istanbul BFT je algoritam **replikacije mašine stanja (State Machine Replication)**. To znači da svaki validator održava identičnu kopiju lanca i prolazi kroz strogo definisana stanja kako bi se osiguralo da svi čvorovi postignu konsenzus o istom bloku. Pre nego što analiziramo detaljne korake, definisaćemo ključna stanja u kojima se validator može nalaziti:

- **NEW ROUND:** Početna tačka. Predlagač treba da pošalje predlog, a ostali validatori čekaju PRE-PREPARE poruku.
- **PRE-PREPARED:** Stanje u kojem je validator primio predlog i emitovao PREPARE poruku. Sada čeka kvorum ($\lceil 2N/3 \rceil$) poruka od ostalih kako bi potvrdio da i drugi vide isti predlog.
- **PREPARED:** Validator je sakupio dovoljno dokaza o pripremi i emituje COMMIT poruku. Ovo je znak da on lično prihvata blok.
- **COMMITTED:** Sakupio je kvorum COMMIT poruka i spreman je da trajno umetne blok u blockchain.
- **FINAL COMMITTED:** Blok je uspešno upisan. Validator je završio posao za tu sekvencu i spreman je za novu rundu.
- **ROUND CHANGE:** Rezervno stanje. Ako se nešto pokvari, validator čeka kvorum ROUND CHANGE poruka kako bi prešao na novog lidera.

State Transitions:



Sada ćemo ova stanja videti u praksi kroz trofazni proces glasanja:

- **Protokol u normalnim okolnostima (Normal Case)**

U idealnim uslovima, kada je lider pošten i mreža stabilna, algoritam prolazi kroz sledeće faze:

1. New Round → Pre-prepared: Predlagač prikuplja transakcije iz bazena transakcija (txpool) i generiše predlog bloka i emituje ga validatorima. Zatim ulazi u PRE-PREPARED stanje.

Svaki validator ulazi u PRE-PREPARED nakon prijema PRE-PREPARE poruke uz sledeće uslove:

1. Predlog bloka je od validnog predlagača.
2. Zaglavlje (header) bloka je validno.
3. Sekvenca i runda predloga bloka se poklapaju sa stanjem validatora.

Validator emituje PREPARE poruku ostalim validatorima.

2. Pre-prepared → Prepared: Validatori su proverili predlog i razmenili dokaze da su ga videli. Čekaju dvotrećinsku većinu PREPARE poruka.

Validator prima $[2N/3]$ validnih PREPARE poruka da bi ušao u PREPARED stanje. Validne poruke su u skladu sa sledećim uslovima:

1. Poklapaju se sekvenca i runda.
2. Poklapa se hash bloka.
3. Poruke su od poznatih validatora.

Validator emituje COMMIT poruku nakon ulaska u PREPARED stanje.

3. Prepared → Committed: Validatori su se složili da je predlog validan. Sada razmenjuju dokaze da će taj blok zaista "zaključati" u bazu.

Validator prima $[2N/3]$ validnih COMMIT poruka da bi ušao u COMMITTED stanje. Validne poruke su u skladu sa sledećim uslovima:

1. Poklapaju se sekvenca i runda.
2. Poklapa se hash bloka.
3. Poruke su od poznatih validatora.

4. Committed → Final Committed: Blok se upisuje u bazu (Insert Block). Ako uspe, ide se u "Final Committed" i čeka se nova runda.

Validator dodaje $[2N/3]$ potpisa potvrde (commitment signatures) u extraData i pokušava da umetne blok u blockchain.

Validator ulazi u FINAL COMMITTED stanje kada umetanje uspe.

Bitno je naglasiti da su blokovi u IBFT-u finalni, što znači da nema forkova (grananja). IBFT nalaže da se grananja ne smeju kreirati, pa čim se većina složi oko bloka, on postaje 'zaključan' (locked in). To znači da nijedan drugi blok neće biti razmatran sve dok se ne završi

pokušaj umetanja ovog bloka u lanac. Dakle, ili je blok uspešno umetnut, ili umetanje ne uspe, blok se odbacuje i predlaže se novi blok na istoj visini lanca.

5. FINAL COMMITTED -> NEW ROUND: Validatori biraju novog predlagača i pokreću tajmer nove runde.

Stanje algoritma

Svaki validator u svakom trenutku održava stanje definisano kroz pet promenljivih:

1. λ_i (Instanca): Identifikuje broj bloka koji se trenutno obrađuje.
2. r_i (Runda): Identifikuje trenutni pokušaj postizanja dogovora (počinje od 1).
3. pr (Pripremljena runda): Najviša runda za koju je validator primio kvorum poruka.
4. pv (Pripremljena vrednost): Vrednost za koju se validator pripremio (inicijalno prazno $\perp$).
5. $inputValue_i$: Vrednost koju validator želi da predloži (txpool transakcije).

TEHNIČKA ANALIZA NORMALNOG RADA (ALGORITMI 1 I 2)

Nakon što smo definisali stanja na dijagramu, analiziraćemo preciznu logiku izvršavanja kroz pseudokod koji rad definiše.

Algorithm 1 IBFT pseudocode for process p_i : constants, state variables, and ancillary procedures

```
1: constants:  
2:    $p_i$                                 ▷ The identifier of the process  
  
3: state variables:  
4:    $\lambda_i$                                 ▷ The identifier of the consensus instance  
5:    $r_i$                                 ▷ The current round  
6:    $pr_i$                                 ▷ The round at which the process has prepared  
7:    $pv_i$                                 ▷ The value for which the process has prepared  
8:    $inputValue_i$                         ▷ The value passed as input to this instance  
  
9: timer:  
10:   $timer_i$   
  
11: procedure START( $\lambda$ ,  $value$ )  
12:    $\lambda_i \leftarrow \lambda$   
13:    $r_i \leftarrow 1$   
14:    $pr_i \leftarrow \perp$   
15:    $pv_i \leftarrow \perp$   
16:    $inputValue_i \leftarrow value$   
17:   if LEADER( $h_i$ ,  $r_i$ ) =  $p_i$  then  
18:     broadcast  $\langle \text{PRE-PREPARE}, \lambda_i, r_i, inputValue_i \rangle$  message  
19:   set  $timer_i$  to running and expire after  $t(r_i)$ 
```

Algorithm 2 IBFT pseudocode for process p_i : normal case operation

```
1: upon receiving a valid  $\langle \text{PRE-PREPARE}, \lambda_i, r_i, \text{value} \rangle$  message  $m$  from  $\text{LEADER}(\lambda_i, \text{round})$  such that  $\text{JUSTIFYPREPREPARE}(m)$  do
2:   set  $\text{timer}_i$  to running and expire after  $t(r_i)$ 
3:   broadcast  $\langle \text{PREPARE}, \lambda_i, r_i, \text{value} \rangle$ 

4: upon receiving a quorum of valid  $\langle \text{PREPARE}, \lambda_i, r_i, \text{value} \rangle$  messages do
5:    $pr_i \leftarrow r_i$ 
6:    $pv_i \leftarrow \text{value}$ 
7:   broadcast  $\langle \text{COMMIT}, \lambda_i, r_i, \text{value} \rangle$ 

8: upon receiving a quorum  $Q_{\text{commit}}$  of valid  $\langle \text{COMMIT}, \lambda_i, \text{round}, \text{value} \rangle$  messages do
9:   set  $\text{timer}_i$  to stopped
10:   $\text{DECIDE}(\lambda_i, \text{value}, Q_{\text{commit}})$ 
```

Inicijalizacija sistema (Algoritam 1)

Za bilo koji ispravan proces (validator) p_i , izvršavanje jedne instance algoritma (označene sa λ) počinje pozivom procedure Start (Algoritam 1, linija 11). Ova procedura uzima identifikator instance λ i početnu vrednost koju validator želi da predloži. U ovom koraku se vrši inicijalizacija promenljivih stanja. Ukoliko je proces p_i izabran za lidera trenutne runde, on odmah emituje (*broadcast*) poruku PRE-PREPARE, predlažući svoju ulaznu vrednost (`inputValue_i`) ostalim učesnicima.

Operacija uobičajenog slučaja (Algoritam 2)

Kada lider pokrene proces, ostali validatori prate logiku **Algoritma 2**, koji opisuje rad u idealnim uslovima (blagovremena komunikacija i ispravan lider):

1. Validacija predloga (Linije 1–3): Nakon što proces p_i primi poruku PRE-PREPARE od lidera za instancu λ_i i trenutnu rundu r_i , on proverava da li poruka nosi "opravdanu vrednost" (pomoću funkcije JUSTIFYPREPREPARE). Ako je uslov ispunjen, proces p_i resetuje svoj tajmer i emituje poruku PREPARE svim ostalim validatorima.

2. Postizanje stanja pripremljenosti (Linije 4–7): Nakon što proces p_i prikupi kvorum ($\lceil 2N/3 \rceil$) validnih PREPARE poruka koje nose istu vrednost za istu instancu i rundu, on ažurira svoje lokalne promenljive:

o **pr_i (prepared round)**: postavlja se na trenutnu rundu r_i .

o **pv_i (prepared value)**: postavlja se na vrednost iz primljenih poruka. U ovom trenutku kažemo da se proces p_i **pripremio** (engl. *prepared*) za datu rundu i vrednost. Odmah nakon toga, proces emituje poruku COMMIT.

3. Donošenje odluke (Linije 8–10): Konačni korak u normalnom radu dešava se kada proces p_i primi kvorum validnih COMMIT poruka. Ove poruke moraju biti usaglašene po rundi i vrednosti. Kada se ovaj uslov ispuni, proces p_i zvanično donosi odluku pozivanjem eksterne funkcije DECIDE. Ovim se instanca λ_i završava uspešno, a blok se smatra finalizovanim.

- Šta kada nesto podje po zlu? (MEHANIZAM PROMENE RUNDE (ALGORITAM 3))

U situacijama kada sistem naiđe na prepreku, aktivira se proces **Round Change**. Ovaj mehanizam je najkompleksniji deo IBFT algoritma jer mora da osigura da se svi poštteni validatori prebace na istu novu rundu bez cepanja mreže. Postoje tri uslova koji bi pokrenuli ROUND CHANGE (promenu runde):

1. **Tajmer istekne (Timeout):** Lider nije poslao predlog na vreme.
2. **Nevalidna PRE-PREPARE poruka:** Lider je poslao pogrešne podatke.
3. **Neuspešno umetanje bloka:** Čak i ako je konsenzus postignut, blok ne može biti upisan u bazu.

Analiza pseudokoda:

Prvi blok (Linije 1-4): Lokalni tajmer ističe

Ovaj deo se aktivira kada čvor primeti da se u trenutnoj rundi ništa ne dešava.

1. **upon *timeri* is expired do:** Kada tajmer čvora p_i za trenutnu rundu istekne.
2. **$ri \leftarrow ri+1$:** Čvor inkrementira (povećava za 1) svoj broj runde.
3. **set *timeri* to running and expire after $t(ri)$:** Pokreće novi tajmer za novu rundu. Vreme čekanja je sada duže (funkcija $t(ri)$).
4. **broadcast $\langle \text{ROUND-CHANGE}, \lambda_i, ri, pri, pvi \rangle$:** Čvor šalje poruku svim ostalima da želi da pređe u rundu ri . Uz to šalje informaciju o tome da li je u nekoj prethodnoj rundi već zaključao neku vrednost (pri, pvi) .

Drugi blok (Linije 5-10): Usklađivanje sa ostalima (F-kolekcija)

Ovaj mehanizam služi da "pogura" čvor koji kasni ili da ubrza proces ako je većina već odlučila da pređe u višu rundu.

1. **upon receiving a set *Frc* of $f+1$ valid messages...:** Kada čvor primi $f+1$ poruka za promenu runde gde su sve te runde r_j veće od njegove trenutne runde ri .
o *Zašto $f+1$?* Pošto ima najviše f neispravnih čvorova, ako dobiješ $f+1$ poruka, bar jedna mora biti od poštenog čvora. To je dokaz da čvor p_i kasni.
2. **let $\langle \text{ROUND-CHANGE}, \lambda_i, rmin, \dots \rangle \in Frc$ such that...:** Pronalazi se najmanja runda ($rmin$) među onima koje su veće od trenutne runde čvora p_i .
3. **(Uslov za $rmin$):** Definiše da je $rmin$ najmanja runda u tom setu poruka.
4. **$ri \leftarrow rmin$:** Čvor "skače" direktno na tu rundu kako bi uhvatio korak sa ostatkom mreže.
5. **set *timeri* to running and expire after $t(ri)$:** Resetuje tajmer za tu novu rundu.
6. **broadcast $\langle \text{ROUND-CHANGE}, \lambda_i, ri, pri, pvi \rangle$:** Čvor šalje svoju poruku o promeni runde za tu novu rundu.

Treći blok (Linije 11-16): Novi lider započinje rundu

Ovo izvršava samo onaj čvor koji je **lider** za novu rundu.

1. upon receiving a quorum Q_{rc} of valid messages such that $LEADER(\lambda_i, r_i) = p_i \dots$: Kada proces p_i vidi da je on lider za novu rundu r_i i sakupi kvorum ($2f+1$) poruka od drugih čvorova koji žele tu rundu. Funkcija JUSTIFYROUNDCHANGE proverava da li su poruke ispravne.

2. if $HIGHESTPREPARED(Q_{rc}) \neq \perp$ then: Proverava se da li je ijedan čvor u kvorumu već pripremio (zaključao) neku vrednost u prethodnim rundama.

3. let v such that $(\dots, v) = HIGHESTPREPARED(Q_{rc})$: Ako postoji takva vrednost, novi lider **mora** da predloži nju. Ovo osigurava bezbednost (da se ne poništi nešto što je skoro bilo dogovoreno).

4. else: Ako niko nije ništa pripremio.

5. let v such that $v = inputValue_i$: Lider može slobodno da predloži svoju novu vrednost (npr. nove transakcije).

6. broadcast $\langle PRE\text{-}PREPARE, \lambda_i, r_i, v \rangle$: Lider šalje PRE-PREPARE poruku, čime zvanično započinje fazu glasanja u novoj rundi.

Četvrti blok (Linije 17-18): Pomoć čvorovima koji su "ispali" iz toka

Ovo se dešava ako je konsenzus za instancu λ_i već postignut, ali neko i dalje šalje zahteve za promenu runde.

1. upon receiving a valid $ROUND\text{-}CHANGE \dots \wedge p_i$ has decided...: Ako čvor p_i primi poruku za promenu runde, a on je već doneo odluku (završio taj blok/instancu) pozivajući funkciju DECIDE.

2. send Q_{commit} to process p_j : Čvor p_i šalje dokaz o završenom konsenzusu (Q_{commit}) čvoru p_j . To pomaže čvoru p_j da odmah pređe na sledeći blok umesto da uzalud pokušava da menja rundu.

Rezime logike

IBFT round change osigurava:

1. Liveness (Živost): Ako sistem stane, tajmer će naterati čvorove da pređu u sledeću rundu (Linije 1-4).

2. Synchronization (Sinhronizacija): Čvorovi koji kasne brzo sustižu ostale (Linije 5-10).

3. Safety (Bezbednost): Novi lider ne može da predloži bilo šta, već mora da poštuje vrednosti koje su u prethodnim pokušajima došle blizu potvrde (Linije 11-16).

Algorithm 3 IBFT pseudocode for process p_i : round changes

```
1: upon  $timer_i$  is expired do
2:    $r_i \leftarrow r_i + 1$ 
3:   set  $timer_i$  to running and expire after  $t(r_i)$ 
4:   broadcast  $\langle \text{ROUND-CHANGE}, \lambda_i, r_i, pr_i, pv_i \rangle$ 

5: upon receiving a set  $F_{rc}$  of  $f + 1$  valid  $\langle \text{ROUND-CHANGE}, \lambda_i, r_j, -, - \rangle$  messages such
   that  $\forall \langle \text{ROUND-CHANGE}, \lambda_i, r_j, -, - \rangle \in F_{rc} : r_j > r_i$  do
6:   let  $\langle \text{ROUND-CHANGE}, h_i, r_{min}, -, - \rangle \in F_{rc}$  such that:
7:    $\forall \langle \text{ROUND-CHANGE}, \lambda_i, r_j, -, - \rangle \in F_{rc} : r_{min} \leq r_j$ 
8:    $r_i \leftarrow r_{min}$ 
9:   set  $timer_i$  to running and expire after  $t(r_i)$ 
10:  broadcast  $\langle \text{ROUND-CHANGE}, \lambda_i, r_i, pr_i, pv_i \rangle$ 

11: upon receiving a quorum  $Q_{rc}$  of valid  $\langle \text{ROUND-CHANGE}, \lambda_i, r_i, -, - \rangle$  messages such
    that  $\text{LEADER}(\lambda_i, r_i) = p_i \wedge \text{JUSTIFYROUNDCHANGE}(Q_{rc})$  do
12:  if  $\text{HIGHESTPREPARED}(Q_{rc}) \neq \perp$  then
13:    let  $v$  such that  $(-, v) = \text{HIGHESTPREPARED}(Q_{rc})$ 
14:  else
15:    let  $v$  such that  $v = \text{inputValue}_i$ 
16:  broadcast  $\langle \text{PRE-PREPARE}, \lambda_i, r_i, v \rangle$ 

    ▷ We can omit this if we assume some mechanism external to the consensus algorithm that ensures
    synchronization of decided values.

17: upon receiving a valid  $\langle \text{ROUND-CHANGE}, \lambda_i, -, -, - \rangle$  message from  $p_j \wedge p_i$  has decided
    by calling  $\text{DECIDE}(\lambda_i, -, Q_{commit})$  do
18:  send  $Q_{commit}$  to process  $p_j$ 
```

TREĆI DEO: NAPREDNI KONCEPTI

Mehanizam opravdanja (Message Justification)

Ono što ovaj rad posebno ističe je mehanizam opravdanja poruka. Ovo je 'tajni sastojak' koji sprečava malicioznog lidera u novoj rundi da predloži nešto što bi poništilo ono što su čvorovi skoro prihvatili u prethodnoj rundi. Svaka PRE-PREPARE poruka mora biti 'opravdana' dokazom o prethodnim kvorumima. To osigurava da, ako je bilo koji pošten čvor već 'zaključan' na neku vrednost, novi lider mora da predloži tu istu vrednost. Ovo direktno rešava problem 'safety' povreda koje su mučile ranije verzije.

1. Funkcija $\text{HIGHESTPREPARED}(Q_{rc})$ (Linije 5-6)

Ovu funkciju je najlakše prvo objasniti jer je ostale koriste. Ona služi da iz gomile poruka izvuče onu koja je "najdalje stigla".

Linija 5: Definiše funkciju koja prima kvorum poruka o promeni runde.

Linija 6: Kaže: "Nađi mi par (runda, vrednost) takav da **postoji** barem jedna poruka u kvorumu sa tom rundom (pr) i vrednošću (pv), ali tako da je ta runda pr **veća ili jednaka** od svih ostalih rundi (pr_j) u tom kvorumu."

2. Predikat $\text{JUSTIFYROUNDCHANGE}(Q_{rc})$ (Linije 1-2)

Ovo proverava da li je kvorum poruka za promenu runde koji je lider sakupio uopšte validan.

Linija 2: Postoje dva načina da promena runde bude opravdana (povezana sa "ILI"):

o **Prvi uslov:** $\forall \langle \dots \rangle \in Q_{rc} : pr_j = \perp \wedge pv_j = \perp$: Svi u kvorumu kažu: "Nismo ništa pripremili u prethodnim rundama". (Sistem kreće od nule).

o **Drugi uslov:** Postoji kvorum validnih PREPARE poruka za vrednost koja je "najviša pripremljena" (HIGHESTPREPARED) - Ako je neko nešto ipak pripremio, moramo imati dokaz (kvorum PREPARE poruka) da je ta specifična vrednost zaista bila legalno pripremljena.

3. Predikat JUSTIFYPREPARE(poruka) (Linije 3-4)

Ovo je najbitniji deo koji izvršavaju obični čvorovi kada im novi lider pošalje predlog (PRE-PREPARE). Oni ne veruju lideru na reč, već pozivaju ovu funkciju.

Linija 4: Predlog lidera je opravdan ako važi jedno od dva pravila:

o **Pravilo 1:** *round*=1 - Ako je ovo prva runda (runda 0 je propala, sad smo u rundi 1), lider može da predloži šta želi jer još ništa nije moglo biti "zaključano". (Napomena: u nekim verzijama je ovo runda 0, zavisno od brojanja).

o **Pravilo 2:** Lider je priložio kvorum *Q_{rc}* (dokaz) koji pokazuje:

- **ILI** su svi u tom dokazu rekli da nemaju ništa pripremljeno ($\perp$).
- **ILI** vrednost koju lider sada predlaže (value) je **identična** onoj vrednosti koja je bila "najviša pripremljena" u tom dokazu.

Rezime:

Ovaj kod osigurava **Safety (Bezbednost)**. On garantuje da jednom kada sistem "zagriže" neku vrednost (pripremi je), nijedan sledeći lider ne može da je promeni, čak i ako je taj lider zlonameran, jer mu ostali čvorovi neće "opravdati" (justify) poruku ako nema validan dokaz iz prethodnih rundi.

Algorithm 4 IBFT pseudocode for process p_i : message justification

```

1: predicate JUSTIFYROUNDCHANGE( $Q_{rc}$ )
2:   return
       $\forall \langle \text{ROUND-CHANGE}, \lambda_i, r_i, pr_j, pv_j \rangle \in Q_{rc} : pr_j = \perp \wedge pv_j = \perp$ 
       $\vee$  received a quorum of valid  $\langle \text{PREPARE}, \lambda_i, pr, pv \rangle$  messages such that:
           $(pr, pv) = \text{HIGHESTPREPARED}(Q_{rc})$ 

3: predicate JUSTIFYPREPARE( $\langle \text{PRE-PREPARE}, \lambda_i, round, value \rangle$ )
4:   return
       $round = 1$ 
       $\vee$  received a quorum  $Q_{rc}$  of valid  $\langle \text{ROUND-CHANGE}, \lambda_i, round, pr_j, pv_j \rangle$  messages
          such that:
               $\forall \langle \text{ROUND-CHANGE}, \lambda_i, round, pr_j, pv_j \rangle \in Q_{rc} : pr_j = \perp \wedge pr_j = \perp$ 
               $\vee$  received a quorum of valid  $\langle \text{PREPARE}, \lambda_i, pr, value \rangle$  messages such that:
                   $(pr, value) = \text{HIGHESTPREPARED}(Q_{rc})$ 

     $\triangleright$  Helper function that returns a tuple  $(pr, pv)$  where  $pr$  and  $pv$  are, respectively, the prepared round
    and the prepared value of the ROUND-CHANGE message in  $Q_{rc}$  with the highest prepared round
5: function HIGHESTPREPARED( $Q_{rc}$ )
6:   return  $(pr, pv)$  such that:
       $\exists \langle \text{ROUND-CHANGE}, \lambda_i, round, pr, pv \rangle \in Q_{rc} :$ 
       $\forall \langle \text{ROUND-CHANGE}, \lambda_i, round, pr_j, pv_j \rangle \in Q_{rc} : pr_j = \perp \vee pr \geq pr_j$ 

```

TEHNIČKO "PAKOVANJE" BLOKA (Hashing i struktura)

Nakon što su validatori kroz Algoritme 2 i 3 postigli dogovor (faza COMMIT), blok mora biti trajno upisan u bazu. Međutim, ovde se javlja specifičan problem koji IBFT rešava na drugačiji način od običnog Ethereum.

1. Problem "kokoške i jajeta" (Sealing & Hashing)

U Bitcoinu, dokaz da je blok validan je "Hash" (broj sa puno nula). U IBFT-u, dokaz su **digitalni potpisi** validatora. Tu nastaje logički problem:

- Da bi blok bio validan, on u sebi mora da sadrži potpise validatora (kao dokaz kvoruma).
- Ako ubaciš potpise u blok, menjaš sadržaj bloka.
- Ako se promeni sadržaj, menja se i **Hash** bloka.
- Ako se promeni Hash, oni potpisi koje si maločas prikupio više ne važe, jer su oni potpisali "stari" Hash.

Kako IBFT ovo rešava?

Proces se odvija u tri strogo definisana koraka:

1. **Pečat predlagrača (Proposer Seal):** Predlagrač prvi potpisuje zaglavlje bloka, ali dok su polja za ostale potpise **prazna**. To je inicijalni pečat.

2. **Dosledan Hash bloka (Block Hash):** Ovo je ključna inovacija. Prilikom računanja Hash-a bloka, polja sa potpisima ostalih validatora (CommittedSeals) se **ignorišu** (tretiraju se kao prazna).

Ovo je bitno jer Validator A može da prikupi potpise od banaka 1, 2 i 3, a Validator B od banaka 2, 3 i 4. Obojica imaju kvorum, ali njihove liste potpisa su različite. Da se ovi potpisi uključuju u Hash, njih dvojica bi imali različite blokove! Ovako, obojica dobijaju **isti Hash**, bez obzira na to čije su potpise prikupili.

3. **Konačni dokaz (Consensus Proof):** Tek kada se dobije taj fiksni Hash, svako ga potpisuje. Ti potpisi se onda "prilepe" u polje extraData i blok se šalje u bazu.

2. Prilagođavanje zaglavlja: "Nove funkcije u starim poljima"

IBFT je dizajniran da radi unutar Ethereum ekosistema. Da ne bi morali da menjaju celu arhitekturu Ethereum, autori su odlučili da **preimenuju** postojeća polja u zaglavlju bloka koja su se ranije koristila za rudarenje (PoW):

- **ExtraData (Ranac bloka):** Pošto je ovo jedino fleksibilno polje, ono postaje najvažnije. U njega se pakuje sve: lista validatora, potpis predlagrača i svi prikupljeni potpisi kvoruma. To čini svaki blok **samoverifikujućim** – svako ko skine blok može odmah da vidi potpise i zna da je on prošao IBFT konsenzus bez provere cele istorije.
- **Nonce (Glasačka kutija):** Umesto da bude nasumičan broj za rudarenje, Nonce se koristi za glasanje. Ako validatori žele da izbace nekoga iz mreže ili ubace novog člana, oni to "upisuju" u ovo polje.
- **Difficulty i MixHash:** Pošto nema rudarenja, ova polja postaju "markeri". Difficulty je uvek 1, a MixHash nosi specijalan "magični broj" koji klijentima (poput Metamask-a) govori: *"Ovo je Istanbul blok, ne traži rudare ovde"*.

DOKAZI ISPRAVNOSTI I ANALIZA PERFORMANSI

Da bi algoritam konsenzusa bio smatran pouzdanim, on mora rigorozno dokazati tri ključne osobine: **Agreement** (Dogovor), **Validity** (Validnost) i **Termination** (Terminacija). Rad koji analiziramo koristi Lamportovu strukturu dokaza kako bi potvrdio da IBFT ispunjava ove uslove čak i u prisustvu zlonamernih aktera.

Tri stuba ispravnosti IBFT-a

1. Dogovor (**Agreement**):

o *Definicija*: Ako jedan ispravan proces odluči o vrednosti v , nijedan drugi ispravan proces ne sme doneti odluku o različitoj vrednosti v' .

o *Mehanizam*: Unutar jedne runde, ovo se garantuje matematikom kvoruma ($2F+1$). Nemoguće je formirati dva različita kvoruma u mreži od $3F+1$ čvorova. Kroz različite runde, sigurnost čuva mehanizam "zaključavanja" i opravdanja (Justification). Jednom kada je proces odluke počeo, mreža je "zaključana" za tu vrednost sve dok se blok ne upiše.

2. Validnost (**Validity**):

o *Definicija*: Ako ispravan proces donese odluku o vrednosti v , ta vrednost mora zadovoljiti eksterni predikat β (mora biti ispravna prema pravilima aplikacije).

o *Mehanizam*: Svaki validator pre slanja poruka (PREPARE ili COMMIT) koristi funkciju provere β . On verifikuje potpise transakcija i ispravnost stanja. Ako lider pokuša da podmetne nevalidan blok, ispravni čvorovi će odbiti glasanje, čime je nemoguće prikupiti kvorum.

3. Terminacija (**Termination**):

o *Definicija*: Svaki ispravan proces na kraju mora doneti odluku.

o *Mehanizam*: IBFT koristi tajmere i proces promene runde kako bi izbegao "zaglavljivanje" (deadlock). Algoritam podrazumeva **GST (Global Settlement Time)** – pretpostavku da će mreža nakon perioda nestabilnosti postati sinhrona dovoljno dugo da ispravan lider završi proces. Takođe, mehanizam "sustizanja" (catch-up) omogućava čvorovima koji kasne da prime dokaze o odluci od ostalih i nastave sa radom.

Poredjenje performansi

	Communication Complexity		Latency
	Normal Case	View Change	Message Delays
DLS [13]	$O(n^4)$	$O(n^4)$	$O(n)$
PBFT [7]	$O(n^2)$	$O(n^3)$	3
Zyzyva [16]	$O(n)$	$O(n^3)$	1 / 3*
Spinning [27]	$O(n^2)$	$O(n^3)$	3
SBFT [15]	$O(n)^{\dagger}$	$O(n^2)$	5 / 7
HotStuff [28]	$O(n)$	$O(n)$	8
IBFT (Sec. 4)	$O(n^2)$	$O(n^2)$	3

Table 1: Performance of related algorithms when communication is timely. Message delays for dual-mode protocols are shown as x / y where x is for the optimistic environment and y otherwise.

Kada uporedimo IBFT sa drugim vodećim algoritmima (poput PBFT, HotStuff ili Zyzyva), vidimo jasne prednosti i kompromise:

- **Latencija (Message Delays):** IBFT postiže terminaciju u samo **3 kašnjenja poruka**. To ga čini značajno bržim od modernih rešenja poput **HotStuff-a**, kojem je potrebno čak 8 kašnjenja. U rangu je sa klasičnim PBFT-om, ali uz optimizovaniju komunikaciju.
- **Komunikaciona složenost:** IBFT ima složenost od **$O(N^2)$** (kvadratna složenost) u oba slučaja – i tokom normalnog rada i tokom promene runde (View Change).

Ove karakteristike čine IBFT idealnim za mreže srednje veličine (do par stotina validatora), što je standard za većinu konzorcijumskih i poslovnih blockchain rešenja.

III Arhitektura implementiranog simulatora

Simulator je napisan u jeziku Go, koristeći gorutine za simulaciju paralelnog rada čvorova i kanale (chan) za simulaciju mrežne komunikacije.

Ključne komponente:

- IBFTNode: Struktura koja čuva stanje validatora:
- Lambda (λ): Visina bloka (Sequence).
- Round (r): Trenutni pokušaj postizanja dogovora.
- PR / PV: Memorija validatora (Prepared Round / Prepared Value) – ključna za sigurnost pri promeni runde.
- MsgChan: Ulazno sanduče za poruke.

- IBFTMessage: Struktura poruke koja sadrži tip (Pre-Prepare, Prepare, Commit, Round-Change), metapodatke i ProofRC (dokaz o opravdanosti runde).

Analiza simuliranih scenarija

Scenario 1: Normalno izvršavanje (Ideal Case)

- Opis: Svi čvorovi su ispravni, lider je pošten.
- Mehanika: Mreža prolazi kroz sve tri faze (Pre-prepare -> Prepare -> Commit).
- Cilj: Dokazati da sistem postiže kvorum od $2F+1$ (3 od 4 čvora) i uspešno finalizuje blok.
- Rezultat: Blokovi se nižu (Lambda 1, 2, 3...) uz rotaciju lidera (Round-Robin).

Scenario 2: Otkaz lidera (Liveness Test)

- Opis: Lider (Node 1) se gasi na samom početku.
- Mehanika: Čvorovi ne dobijaju predlog, njihovi tajmeri ističu (Timeout).
- Cilj: Demonstracija Round Change mehanizma.
- Rezultat: Čvorovi šalju ROUND-CHANGE poruke, sinhronizuju se na osnovu $F+1$ pravila i biraju novog lidera u Rundi 2 koji uspešno završava blok.

Scenario 3 & 4: Gubitak kvoruma (Safety Test)

- Opis: Više od F čvorova (u ovom slučaju 2 čvora) otkáže pre ili tokom glasanja.
- Mehanika: Preostala 2 čvora pokušavaju da glasaju, ali dostižu samo $2/3$ glasova.
- Cilj: Dokazati da IBFT preferira Safety nad Liveness-om.
- Rezultat: Sistem se zaustavlja (stalling). Ne donosi se nikakva odluka jer nema kvoruma od 3 čvora. Ovo sprečava "split-brain" situaciju.

Scenario 5: Tehnička greška (Database/Application Error)

- Opis: Konsenzus je postignut, ali upis u bazu podataka ne uspeva.
- Mehanika: Čvorovi dobijaju $2F+1$ COMMIT poruka, ali simulirani upis vraća grešku.
- Cilj: Razgraničenje mrežnog dogovora od finalizacije podataka.
- Rezultat: Čvorovi odbijaju da pređu na sledeći blok (Lambda) dok se trenutni ne upiše. Mreža se vrti u rundama pokušavajući ponovo, čime se čuva integritet lanca.

Scenario 6: Vizantijski lider (Prevara / Splitting)

- Opis: Lider je maliciozan i šalje različite predloge različitim čvorovima (Node 0 dobije vrednost X, Node 2 dobije vrednost Y).
- Mehanika: Čvorovi šalju PREPARE glasove za ono što su primili.
- Cilj: Dokazati da lider ne može samostalno da nametne odluku ako ne postoji kvorum za istu vrednost.
- Rezultat: Pošto je mreža podeljena (npr. 2 glasa za X, 2 za Y), niko ne dostiže kvorum od 3 glasa. Runda propada, tajmer ističe i poštenu lider u sledećoj rundi oporavlja mrežu.

Scenario 7: Mehanizam opravdanja (Justification / Safety Lock)

- Opis: Najkompleksniji scenario. Lider u Rundi 2 pokušava da promeni vrednost bloka koja je već bila u procesu odobravanja (ili da podmetne nešto treće bez dokaza).
- Mehanika: Validator koristi funkciju JustifyPrePrepare.
- Cilj: Testiranje dokaza o promeni runde (ProofRC).
- Rezultat: Čvorovi analiziraju skup ROUND-CHANGE poruka. Kada lider predloži ZLONAMERNU VREDNOST, validatori je odbijaju jer dokaz pokazuje da lider nema pravo da je predloži. Ovo je ključna odbrana od "Safety" napada u višim rundama.

Ključni algoritmi u kodu

- **IsLeader (Round-Robin selekcija)**

Lider se menja deterministički na osnovu visine bloka i runde: **ID = (Lambda + Round) % BrojValidator**. Ovo osigurava da nijedan maliciozni čvor ne može trajno blokirati mrežu.

- **HandleMessage (State Machine)**

Čvor je implementiran kao mašina stanja koja reaguje na poruke. Implementirane su stroge provere:

1. Unique Votes: Čvor broji samo unikatne glasove validatora za određenu vrednost.
2. Safety Locking: Kada se dostigne kvorum za PREPARE, čvor "zaključava" vrednost u promenljive PR i PV.
3. Quorum Logic: Za odluku je uvek potrebno $2F+1$ glasova, dok je za promenu runde dovoljno $F+1$ (da bi se ubrzao oporavak ako bar jedan pošten čvor vidi problem).

- **Timer (Exponential Backoff)**

Trajanje tajmera se duplira sa svakom rundom: $5s * 2^{(\text{Round}-1)}$. Ovo sprečava mrežu da upadne u beskonačne promene rundi kod sporih internet veza (Liveness garant).

IV Zaključak

IBFT predstavlja evoluciju klasičnih BFT algoritama prilagođenu modernim blockchain potrebama. On je robustan, deterministički algoritam koji nudi optimalan balans između otpornosti na greške i brzine rada.

Zahvaljujući formalnoj verifikaciji i analizi koju je sproveo **Henrique Moniz (2020)**, dobili smo dokaz da su rane mane IBFT-a (poput gubljenja liveness-a) uspešno rešene. Danas, on nije samo teoretski model, već praktična osnova za sisteme poput **Quorum-a**, koji pokreću kritične finansijske aplikacije širom sveta. Iako postoje algoritmi sa manjim mrežnim saobraćajem, IBFT ostaje jedan od najpouzdanijih izbora tamo gde se zahteva instantna finalnost i visoka sigurnost u konzorcijumskom okruženju.